

Depression Detection Engine - Comprehensive Project Documentation

This document provides a complete overview of the Depression Detection Engine project, along with its full source code.

Main README

File: README.md

🧠 Depression Detection Engine

An AI-powered sentiment analysis and risk assessment platform designed to identify depressive signals in social media threads using state-of-the-art NLP models.

📁 Project Information

Category	Details
Project Title	Depression Detection Engine
Type	Copyrighted Intellectual Property
Current Status	Building (Version 1.0.0)

👥 Team Details

Name	Roll Number
Shivansh Sharma	2210990832
Armaan Sharma	2210990158

🚀 Overview

The **Depression Detection Engine** leverages a local **DistilBERT-based transformer model** (via ONNX Runtime) to analyze user comments across various platforms. Unlike traditional sentiment analysis, this engine is specifically tuned to detect emotional distress, withdrawal signals, and depressive tendencies.

Key Features:

- Local Inference**: Uses `Transformers.js` and ONNX for privacy-preserving, local machine learning.
- Risk Scoring**: Implements a severity-based ranking system (Normal, Mild, Severe).
- Hybrid Backend**: Powered by Node.js Express for low-latency API responses.

- **Modern Frontend**: A premium React interface with real-time analysis visualization.

🔧 Tech Stack

- **Frontend**: React, TypeScript, Vite, Vanilla CSS (Glassmorphism)
- **Backend**: Node.js, Express.js
- **Machine Learning**: Transformers.js, ONNX Runtime, DistilBERT
- **Environment**: Cross-platform (Windows/Mac/Linux)

⚖️ Copyright & License

© 2026 Shivansh Sharma & Team. All Rights Reserved.
This project is protected under academic copyright and is intended for educational purposes only.

📈 Current Status

The project is currently in its **Building Status**. All core modules (ML Engine, Backend API, and Frontend Dashboard) are fully operational and have been validated against simulated social media datasets.

SourceCode README

File: SourceCode/README.md

🧠 Depression Detection Engine – Source Code

This directory contains the full implementation of the Depression Detection Engine.

📁 Project Information

- **Title**: Depression Detection Engine
- **Type**: Copyright
- **Status**: Finalized

👥 Team

- **Shivansh Sharma** (2210990832)
- **[Armaan Sharma]** (2210990158)

📁 Structure

- `/backend-node``: Express.js server and ML services.
- `/frontend``: React/TypeScript user interface.

– `/model-onnx`: Local ONNX weights for DistilBERT.

🚀 Getting Started

1. Install dependencies in both `backend-node` and `frontend`.
2. Start the backend: `npm start` (port 8080).
3. Start the frontend: `npm run dev` (port 5173).

Backend Entry Point

File: `SourceCode/backend-node/server.js`

```
const express = require('express');
const cors = require('cors');
const { resolveComments } = require('./services/inputResolver');
const { analyzeComments } = require('./services/analyzer');

const app = express();
app.use(cors());
app.use(express.json());

app.get('/health', (req, res) => {
  res.json({ status: "healthy", model: "distilbert-base-uncased", backend: "node-express" });
});

app.post('/analyze_post', async (req, res) => {
  try {
    const { post_link, comments: fallbackComments } = req.body;
    let commentsToProcess;

    if (post_link) {
      commentsToProcess = await resolveComments(post_link);
    } else if (fallbackComments && fallbackComments.length > 0) {
      commentsToProcess = fallbackComments;
    } else {
      return res.status(422).json({ detail: "No input provided. Please supply post_link or comments." });
    }

    const result = await analyzeComments(commentsToProcess);

    return res.json({
      processed_comments: result.processed,
      user_risk_rankings: result.user_rankings,
      total_comments: result.processed.length,
      model_version: "1.0.0"
    });
  } catch (e) {
    console.error("Endpoint Error:", e.message);
  }
});
```

```

        if (e.message.includes("No valid comments") ||
e.message.includes("supported")) {
            return res.status(404).json({ detail: e.message });
        }
        res.status(500).json({ detail: e.message });
    }
});

const PORT = process.env.PORT || 8080;
app.listen(PORT, () => {
    console.log(`Node.js Express Server running on port ${PORT}`);
});

```

Analyzer Service

File: SourceCode/backend-node/services/analyzer.js

```

const path = require('path');
require('dotenv').config({ path: path.join(__dirname, '../.env') });
const { predictComment } = require('./modelPredictor');

const sleep = (ms) => new Promise(resolve => setTimeout(resolve, ms));

function generateLocalExplanation(label, topTokens) {
    if (label === "Normal") {
        return "Local Analysis: Comment appears standard with no significant emotional distress indicators detected.";
    }

    const keywords = topTokens.map(t => t.token).join(", ");
    if (keywords) {
        return `Local Analysis: ${label} risk detected. Keywords triggering this classification: [${keywords}].`;
    }

    return `Local Analysis: Classified as ${label} based on linguistic patterns and emotional tone.`;
}

async function analyzeComments(comments) {
    try {
        console.log(`Analyzing ${comments.length} comments locally via Transformers.js...`);
        const processed = [];

        for (const row of comments) {
            const pred = await predictComment(row.comment);
            processed.push({
                username: row.username,
                comment: row.comment,

```

```

        label: pred.label,
        confidence: pred.confidence,
        risk_score: pred.risk_score,
        top_tokens: pred.top_tokens,
        explanation: generateLocalExplanation(pred.label, pred.top_tokens)
    });
}

const userRiskMap = {};
for (const row of processed) {
    if (!userRiskMap[row.username]) {
        userRiskMap[row.username] = { risk_sum: 0, count: 0,
depressive_count: 0 };
    }
    userRiskMap[row.username].risk_sum += row.risk_score;
    userRiskMap[row.username].count += 1;
    if (row.label !== "Normal") {
        userRiskMap[row.username].depressive_count += 1;
    }
}

const userRankings = [];
for (const [username, data] of Object.entries(userRiskMap)) {
    const avgRisk = data.risk_sum / data.count;
    let riskLevel = "Low Risk";
    if (avgRisk > 0.70) riskLevel = "High Risk";
    else if (avgRisk > 0.35) riskLevel = "Medium Risk";

    userRankings.push({
        username,
        average_risk_score: avgRisk,
        risk_level: riskLevel,
        depressive_comment_count: data.depressive_count,
        total_comments: data.count
    });
}
userRankings.sort((a, b) => b.average_risk_score - a.average_risk_score);

return { processed, user_rankings: userRankings };

} catch (e) {
    console.error("Local Node.js Inference failed:", e.message);
    throw e;
}
}

module.exports = { analyzeComments };

```

Input Resolver Service

File: `SourceCode/backend-node/services/inputResolver.js`

```

const axios = require('axios');

async function resolveComments(postLink) {
  if (!postLink.includes("reddit.com")) {
    throw new Error("Only Reddit URLs are currently natively supported for live
scraping.");
  }

  let url = postLink.trim();
  if (url.endsWith("/")) url = url.slice(0, -1);
  if (!url.endsWith(".json")) url += ".json";

  try {
    const response = await axios.get(url, { headers: { 'User-Agent':
'DepressionDetectionNode/1.0' } });
    const commentsData = response.data[1].data.children;
    const res = [];

    for (const item of commentsData) {
      if (item.kind === 't1') {
        const text = item.data.body;
        if (!text || text === '[deleted]' || text === '[removed]' ||
text.length < 5) continue;

        res.push({
          username: item.data.author || "Anonymous",
          comment: text
        });
      }
    }

    if (res.length === 0) {
      throw new Error("No valid comments found in the provided Reddit
thread.");
    }
    return res;
  } catch (e) {
    throw new Error(`Reddit API scraping failed: ${e.message}`);
  }
}

module.exports = { resolveComments };

```

Model Predictor Service

File: SourceCode/backend-node/services/modelPredictor.js

```

const { pipeline, env } = require('@xenova/transformers');
const path = require('path');

```

```

// Disable remote loading, always use local files
env.allowRemoteModels = false;
env.localModelPath = '/';
const MODEL_DIR = path.resolve(__dirname, '../..../model-onnx');

const ID_TO_LABEL = { 0: "Normal", 1: "Mild", 2: "Severe" };
const LABEL_TO_RISK = { "Normal": 0.15, "Mild": 0.55, "Severe": 0.95 };

const DEPRESSION_KEYWORDS = new Set([
  "hopeless", "empty", "worthless", "alone", "tired", "sad", "depressed",
  "anxious", "suicidal", "exhausted", "overwhelmed", "give", "pointless",
  "pain", "hurts", "bed", "avoid", "down", "meh", "drowning", "dark",
  "nobody", "never", "nothing", "stress", "stressed", "hate", "useless",
  "burden", "lost", "fail", "failure", "broken", "numb", "quit"
]);

let classifier = null;

async function getClassifier() {
  if (classifier === null) {
    console.log(`Loading Node.js ONNX DistilBERT Model from: ${MODEL_DIR}`);
    classifier = await pipeline('text-classification', MODEL_DIR, {
      model_file_name: 'model',
      quantized: false
    });
  }
  return classifier;
}

function getKeywordImportance(text, topK = 5) {
  const tokens = text.toLowerCase().split(/[.,!?:;]+/).filter(t => t.length > 0);
  const scores = [];
  for (const token of tokens) {
    if (DEPRESSION_KEYWORDS.has(token)) {
      scores.push({ token, importance: 1.0 });
    }
  }
  return scores.slice(0, topK);
}

function cleanText(text) {
  return text.toLowerCase().trim();
}

async function predictComment(text) {
  const pipe = await getClassifier();
  const processed = cleanText(text);

  // Inference
  const output = await pipe(processed);

```

```

const label = output[0].label;
const confidence = output[0].score;

let finalLabel = label;
if (label.startsWith("LABEL_")) {
  const id = parseInt(label.replace("LABEL_", ""));
  finalLabel = ID_TO_LABEL[id];
}

const riskScore = (LABEL_TO_RISK[finalLabel] || 0) * confidence;
const topTokens = getKeywordImportance(processed);

return {
  label: finalLabel,
  confidence: parseFloat(confidence.toFixed(4)),
  risk_score: parseFloat(riskScore.toFixed(4)),
  top_tokens: topTokens,
  comment: text
};
}

module.exports = { predictComment };

```

Backend Package Info

File: SourceCode/backend-node/package.json

```

{
  "name": "backend-node",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "@xenova/transformers": "^2.17.2",
    "axios": "^1.15.2",
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "nodemon": "^3.1.14",
    "onnxruntime-node": "^1.25.1"
  }
}

```



```
}  
}
```

Frontend Package Info

File: SourceCode/frontend/package.json

```
{  
  "name": "depression-detection-frontend",  
  "version": "1.0.0",  
  "private": true,  
  "type": "module",  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build",  
    "preview": "vite preview"  
  },  
  "dependencies": {  
    "react": "^18.3.1",  
    "react-dom": "^18.3.1"  
  },  
  "devDependencies": {  
    "@types/react": "^18.3.12",  
    "@types/react-dom": "^18.3.1",  
    "@vitejs/plugin-react": "^4.3.4",  
    "typescript": "^5.6.3",  
    "vite": "^5.4.11"  
  }  
}
```

Frontend Vite Config

File: SourceCode/frontend/vite.config.ts

```
import { defineConfig } from "vite";  
import react from "@vitejs/plugin-react";  
  
export default defineConfig({  
  plugins: [react()],  
});
```

Frontend TS Config

File: SourceCode/frontend/tsconfig.json

```
{  
  "compilerOptions": {
```

```
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": false,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "strict": true
  },
  "include": ["src"]
}
```

Frontend HTML

File: SourceCode/frontend/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Depression Detection</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Frontend Main Entry

File: SourceCode/frontend/src/main.tsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

Frontend App Component

File: SourceCode/frontend/src/App.tsx

```
import { FormEvent, useEffect, useState } from "react";
import { CommentTable } from "../components/CommentTable";
import { UserRiskPanel } from "../components/UserRiskPanel";
import { analyzePost } from "../services/api";
import { AnalyzeResponse } from "../types";

function App() {
  const [postLink, setPostLink] = useState("");
  const [commentsJson, setCommentsJson] = useState("");
  const [result, setResult] = useState<AnalyzeResponse | null>(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const submit = async (event: FormEvent) => {
    event.preventDefault();
    setError(null);
    setLoading(true);
    try {
      const parsed = commentsJson.trim() ? JSON.parse(commentsJson) : null;
      const payload: any = {};
      if (postLink.trim().length > 0) payload.post_link = postLink.trim();
      if (parsed) payload.comments = parsed;
      const data = await analyzePost(payload);
      setResult(data);
    } catch (err) {
      const errorMessage = (err as Error).message;
      console.error("API call failed with error:", err);
      // More descriptive error state
      if (errorMessage.includes("No simulated thread found")) {
        setError("Error: No simulated thread found for this post link. Please provide a mock fixture or use the comments fallback.");
      } else {
        setError(`Processing Error: ${errorMessage}`);
      }
    } finally {
      setLoading(false);
    }
  };

  return (
    <main style={{ maxWidth: 1100, margin: "0 auto", padding: "40px 20px" }}>
      <header style={{ textAlign: "center", marginBottom: 40 }} className="animate-fade-in">
        <h1>Depression Detection Engine</h1>
        <p style={{ color: "var(--text-secondary)", fontSize: "1.1rem" }}>
          AI-powered comment thread severity analysis platform.
        </p>
      </header>
    </main>
  );
}
```

```

</header>

<div className="glass-panel animate-fade-in delay-1" style={{ marginBottom: 40
}}>
  <h2 style={{ marginBottom: "0.5rem" }}>Analyze Thread</h2>
  <p style={{ color: "var(--text-secondary)", fontSize: "0.9rem",
marginBottom: "1rem" }}>
    Provide a link to a social media post, or fallback to JSON array of
    comments.
  </p>
  <form onSubmit={submit}>
    <label htmlFor="post-link">Post URL</label>
    <input
      id="post-link"
      value={postLink}
      onChange={(e) => setPostLink(e.target.value)}
      placeholder="https://x.com/sample/post/123"
    />
    <label htmlFor="comments-json">JSON Comments Fallback</label>
    <textarea
      id="comments-json"
      rows={6}
      value={commentsJson}
      onChange={(e) => setCommentsJson(e.target.value)}
    />
    <button type="submit" disabled={loading}>
      {loading ? "Processing NLP Model..." : "Evaluate Content Engine"}
    </button>
  </form>
  {error && <p style={{ color: "var(--color-severe)", marginTop: 16 }}>{error}
</p>}
</div>

{result && (
  <div className="animate-fade-in delay-2">
    <div style={{ display: "flex", justifyContent: "space-between",
alignItems: "flex-end", marginBottom: 20 }}>
      <h2 style={{ margin: 0 }}>Processing Results</h2>
      <div className="badge mild" style={{ fontSize: "0.8rem", padding: "6px
12px" }}>
        API v{result.model_version || "1.0"} • {result.total_comments}
        Comments
      </div>
    </div>

    <UserRiskPanel users={result.user_risk_rankings} />

    <h3 style={{ marginTop: 40, marginBottom: 16 }}>Detailed Breakdown</h3>
    <CommentTable rows={result.processed_comments} />
  </div>
)}
</main>

```

```
);  
}  
  
export default App;
```

Frontend CSS

File: SourceCode/frontend/src/index.css

```
@import url('https://fonts.googleapis.com/css2?  
family=Inter:wght@300;400;500;600;700&display=swap');  
  
:root {  
  --bg-color: #0f111a;  
  --panel-bg: rgba(255, 255, 255, 0.03);  
  --panel-border: rgba(255, 255, 255, 0.08);  
  --text-primary: #f8fafc;  
  --text-secondary: #94a3b8;  
  --accent-color: #3b82f6;  
  --accent-hover: #2563eb;  
  
  --color-normal: #10b981;  
  --color-normal-bg: rgba(16, 185, 129, 0.15);  
  
  --color-mild: #f59e0b;  
  --color-mild-bg: rgba(245, 158, 11, 0.15);  
  
  --color-severe: #f43f5e;  
  --color-severe-bg: rgba(244, 63, 94, 0.15);  
}  
  
* {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
}  
  
body {  
  font-family: 'Inter', sans-serif;  
  background-color: var(--bg-color);  
  background-image:  
    radial-gradient(at 0% 0%, rgba(59, 130, 246, 0.15) 0px, transparent 50%),  
    radial-gradient(at 100% 0%, rgba(244, 63, 94, 0.1) 0px, transparent 50%),  
    radial-gradient(at 100% 100%, rgba(16, 185, 129, 0.05) 0px, transparent 50%);  
  color: var(--text-primary);  
  min-height: 100vh;  
  line-height: 1.6;  
  -webkit-font-smoothing: antialiased;  
}
```

```

h1, h2, h3 {
  font-weight: 600;
  color: var(--text-primary);
}

h1 {
  font-size: 2.5rem;
  margin-bottom: 0.5rem;
  background: linear-gradient(to right, #fff, #94a3b8);
  background-clip: text;
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  letter-spacing: -0.02em;
}

/* Glass panel styling */
.glass-panel {
  background: var(--panel-bg);
  border: 1px solid var(--panel-border);
  backdrop-filter: blur(12px);
  border-radius: 16px;
  padding: 24px;
  box-shadow: 0 4px 30px rgba(0, 0, 0, 0.1);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.glass-panel:hover {
  transform: translateY(-2px);
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
}

/* Inputs and Forms */
input, textarea {
  width: 100%;
  background: rgba(0, 0, 0, 0.2);
  border: 1px solid var(--panel-border);
  color: var(--text-primary);
  padding: 12px 16px;
  border-radius: 8px;
  font-family: inherit;
  font-size: 0.95rem;
  transition: all 0.2s;
  outline: none;
}

input:focus, textarea:focus {
  border-color: var(--accent-color);
  box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.2);
}

input::placeholder, textarea::placeholder {
  color: var(--text-secondary);
}

```

```

}

label {
  display: block;
  font-size: 0.875rem;
  font-weight: 500;
  color: var(--text-secondary);
  margin-bottom: 8px;
  margin-top: 16px;
}

button {
  background: var(--accent-color);
  color: white;
  border: none;
  padding: 12px 24px;
  border-radius: 8px;
  font-weight: 600;
  font-size: 1rem;
  cursor: pointer;
  transition: all 0.2s ease;
  width: 100%;
  margin-top: 24px;
  box-shadow: 0 4px 14px rgba(59, 130, 246, 0.3);
}

button:hover:not(:disabled) {
  background: var(--accent-hover);
  transform: translateY(-1px);
  box-shadow: 0 6px 20px rgba(59, 130, 246, 0.4);
}

button:active:not(:disabled) {
  transform: translateY(1px);
}

button:disabled {
  opacity: 0.7;
  cursor: not-allowed;
  background: #475569;
  box-shadow: none;
}

/* Tables */
.table-wrapper {
  overflow-x: auto;
  border-radius: 12px;
  border: 1px solid var(--panel-border);
  background: rgba(0, 0, 0, 0.2);
}

table {

```

```

width: 100%;
border-collapse: collapse;
text-align: left;
}

th {
  background: rgba(255, 255, 255, 0.05);
  padding: 16px;
  font-size: 0.875rem;
  font-weight: 600;
  color: var(--text-secondary);
  border-bottom: 1px solid var(--panel-border);
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

td {
  padding: 16px;
  border-bottom: 1px solid var(--panel-border);
  font-size: 0.95rem;
}

tr:last-child td {
  border-bottom: none;
}

tbody tr {
  transition: background-color 0.2s;
}

tbody tr:hover {
  background-color: rgba(255, 255, 255, 0.03);
}

/* Badges */
.badge {
  display: inline-flex;
  align-items: center;
  padding: 4px 10px;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.badge.normal {
  background: var(--color-normal-bg);
  color: var(--color-normal);
  border: 1px solid rgba(16, 185, 129, 0.3);
}

```



```
.badge.mild {
  background: var(--color-mild-bg);
  color: var(--color-mild);
  border: 1px solid rgba(245, 158, 11, 0.3);
}

.badge.severe {
  background: var(--color-severe-bg);
  color: var(--color-severe);
  border: 1px solid rgba(244, 63, 94, 0.3);
}

/* Info Cards */
.risk-card {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 20px;
  border-radius: 12px;
  margin-bottom: 12px;
  border: 1px solid var(--panel-border);
  background: rgba(0, 0, 0, 0.2);
  transition: all 0.2s;
  position: relative;
  overflow: hidden;
}

.risk-card::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  bottom: 0;
  width: 4px;
}

.risk-card.low::before { background: var(--color-normal); }
.risk-card.medium::before { background: var(--color-mild); }
.risk-card.high::before { background: var(--color-severe); }

.risk-card:hover {
  transform: translateX(4px);
  background: rgba(255, 255, 255, 0.03);
}

.user-meta {
  display: flex;
  flex-direction: column;
}

.user-name {
  font-weight: 600;
}
```

```

    font-size: 1.1rem;
    margin-bottom: 4px;
  }

.user-stats {
  font-size: 0.85rem;
  color: var(--text-secondary);
}

/* Animations */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

.animate-fade-in {
  animation: fadeIn 0.4s ease forwards;
}

.delay-1 { animation-delay: 0.1s; }
.delay-2 { animation-delay: 0.2s; }

```

Frontend CommentTable Component

File: SourceCode/frontend/src/components/CommentTable.tsx

```

import { CommentPrediction } from "../types";

type Props = { rows: CommentPrediction[] };

export function CommentTable({ rows }: Props) {
  const getBadgeClass = (label: string) => {
    switch (label) {
      case "Normal": return "badge normal";
      case "Mild": return "badge mild";
      case "Severe": return "badge severe";
      default: return "badge";
    }
  };

  return (
    <div className="table-wrapper glass-panel" style={{ padding: 0, overflow: "auto" }}>
      <table>
        <thead>
          <tr>
            <th style={{ width: "10%" }}>User</th>
            <th style={{ width: "35%" }}>Comment</th>
            <th style={{ width: "30%" }}>Reasoning Insights</th>
            <th style={{ width: "10%" }}>Risk Score</th>

```

```

        <th style={{ width: "8%" }}>Label</th>
        <th style={{ width: "7%" }}>Conf.</th>
      </tr>
    </thead>
    <tbody>
      {rows.map((row, idx) => (
        <tr key={`_${row.username}-${idx}`}>
          <td style={{ fontWeight: 500 }}>@{row.username}</td>
          <td style={{ color: "var(--text-secondary)", lineHeight: 1.5 }}>
            {row.comment}
          </td>
          <td>
            {row.top_tokens && row.top_tokens.length > 0 && (
              <div style={{ display: "flex", flexWrap: "wrap", gap: 6,
marginBottom: 6 }}>
                {row.top_tokens.map((tt, i) => (
                  <span key={i} style={{
                    fontSize: "0.75rem",
                    background: "rgba(255,255,255,0.06)",
                    padding: "2px 8px",
                    borderRadius: "12px",
                    border: "1px solid rgba(255,255,255,0.1)",
                    color: "var(--text-primary)",
                    display: "flex",
                    gap: "4px",
                    alignItems: "center"
                  }}>
                    {tt.token}
                    <span style={{ opacity: 0.5, fontSize: "0.7rem",
fontVariantNumeric: "tabular-nums" }}>
                      {tt.importance.toFixed(2)}
                    </span>
                  </span>
                ))}
              </div>
            )}
            {row.explanation && (
              <div style={{ fontSize: "0.80rem", color: "var(--text-secondary)",
fontStyle: "italic", borderLeft: "2px solid var(--accent-color)", paddingLeft: 8 }}>
                {row.explanation}
              </div>
            )}
          </td>
          <td>
            <div style={{ display: "flex", alignItems: "center", gap: "8px" }}>
              <div style={{ flex: 1, background: "rgba(255,255,255,0.1)",
height: "6px", borderRadius: "3px", overflow: "hidden" }}>
                <div
                  style={{
                    height: "100%",
                    width: `${Math.min(row.risk_score * 100, 100)}%`,
                    background: row.label === "Severe" ? "var(--color-severe)" :

```

```

row.label === "Mild" ? "var(--color-mild)" : "var(--color-normal)",
      transition: "width 1s ease-out"
    }}
  />
</div>
<span style={{ fontSize: "0.85rem", fontVariantNumeric: "tabular-
nums" }}>
    {(row.risk_score * 100).toFixed(1)}
  </span>
</div>
</td>
<td>
  <span className={getBadgeClass(row.label)}>{row.label}</span>
</td>
<td style={{ fontVariantNumeric: "tabular-nums", fontSize: "0.9rem"
}}>
    {(row.confidence * 100).toFixed(0)}%
  </td>
</tr>
  )}
</tbody>
</table>
</div>
);
}

```

Frontend UserRiskPanel Component

File: SourceCode/frontend/src/components/UserRiskPanel.tsx

```

import { UserRiskProfile } from "../types";

type Props = { users: UserRiskProfile[] };

export function UserRiskPanel({ users }: Props) {
  const getRiskColorClass = (riskLevel: string) => {
    switch (riskLevel) {
      case "High Risk": return "high";
      case "Medium Risk": return "medium";
      case "Low Risk": return "low";
      default: return "";
    }
  };

  const getBadgeClass = (riskLevel: string) => {
    switch (riskLevel) {
      case "High Risk": return "badge severe";
      case "Medium Risk": return "badge mild";
      case "Low Risk": return "badge normal";
      default: return "badge";
    }
  };
}

```

```

    }
  };

  return (
    <div className="glass-panel" style={{ padding: "20px 24px" }}>
      <h3 style={{ marginBottom: "16px", display: "flex", alignItems: "center", gap: "8px" }}>
        User Aggregation Profiles
        <span style={{ background: "rgba(255,255,255,0.1)", padding: "2px 8px", borderRadius: "12px", fontSize: "0.8rem", color: "var(--text-secondary)" }}>
          {users.length} Users Found
        </span>
      </h3>
      <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fill, minmax(300px, 1fr))", gap: "16px" }}>
        {users.map((user) => {
          const colorClass = getRiskColorClass(user.risk_level);
          return (
            <div key={user.username} className={`risk-card ${colorClass}`}>
              <div className="user-meta">
                <div className="user-name">@{user.username}</div>
                <div className="user-stats">
                  Avg Risk: {(user.average_risk_score * 100).toFixed(1)}
                  <br />
                  Depressive Comments: <strong style={{ color: "var(--text-primary)" }}>{user.depressive_comment_count}</strong> / {user.total_comments}
                </div>
              </div>
            </div>
          );
        })}
      </div>
    </div>
  );
}

```

Frontend API Service

File: SourceCode/frontend/src/services/api.ts

```

import { AnalyzeResponse } from "../types";

const API_BASE = import.meta.env.VITE_API_BASE_URL || "http://localhost:8080";

export async function analyzePost(payload: { post_link?: string; comments?: { username: string; comment: string }[] }): Promise<AnalyzeResponse> {

```

```
const response = await fetch(`${API_BASE}/analyze_post`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(payload),
});

if (!response.ok) {
  const message = await response.text();
  throw new Error(message || "Analyze request failed.");
}

return response.json();
}
```

Frontend Types

File: SourceCode/frontend/src/types/index.ts

```
export type TokenImportance = {
  token: string;
  importance: number;
};

export type CommentPrediction = {
  username: string;
  comment: string;
  label: "Normal" | "Mild" | "Severe";
  confidence: number;
  risk_score: number;
  top_tokens: TokenImportance[];
  explanation: string;
};

export type UserRiskProfile = {
  username: string;
  average_risk_score: number;
  depressive_comment_count: number;
  total_comments: number;
  risk_level: "Low Risk" | "Medium Risk" | "High Risk";
};

export type AnalyzeResponse = {
  processed_comments: CommentPrediction[];
  user_risk_rankings: UserRiskProfile[];
  model_version: string;
  total_comments: number;
};
```
